

Vendor Risk Scoring Management Project

Nazlıcan Aka

Jun 15 2026

1 Tech Stack & Architecture

The project is built following **Clean Architecture** principles, ensuring that dependencies point inwards. This guarantees that changes to databases or external frameworks do not affect core business rules.

- **Backend Framework:** .NET 8 (ASP.NET Core Web API)
- **Database & ORM:** PostgreSQL & Entity Framework Core (Code-First Migration & Seeder)
- **Caching:** Redis (Distributed Cache for high-performance risk evaluation lookups)
- **Logging & Monitoring:** Serilog (JSON Sink) paired with the ELK Stack (Elasticsearch, Logstash, Kibana).
- **Testing:** xUnit, Moq (Unit Testing)
- **Containerization:** Docker & Docker Compose (Multi-container orchestration)

1.1 Architectural Layers

1. **Domain Layer** (`VendorRiskScoring.Domain`): The core of the application with zero external framework dependencies. Contains core entities (`Vendor`, `RiskAssessment`) and domain models.
2. **Application Layer** (`VendorRiskScoring.Application`): Houses the business logic and orchestration. Depends solely on the Domain layer (e.g., `RiskEngineService`, risk rules, factories, and repository interfaces).
3. **Infrastructure Layer** (`VendorRiskScoring.Infrastructure`): Deals with persistence and external services (Entity Framework Core PostgreSQL integration, repository implementations, and JSON matrix providers).
4. **API Layer** (`VendorRiskScoring.API`): The presentation layer acting as the entry point for clients (Controllers, DI setup, Serilog, and Redis configurations).

2 Design Patterns

To manage complexity and promote maintainability, industry-standard design patterns are utilized:

- **Strategy Pattern:** Implemented via `IRiskRule` and rules like `FinancialRiskRule` to decouple individual risk calculation algorithms.
- **Factory Pattern:** Encapsulated in `RiskAssessmentFactory` to translate raw scores into risk levels (`Critical`, `High`, `Medium`, `Low`) and human-readable explanations.
- **Repository Pattern:** Abstracts data persistence through `IVendorRepository` and `EfVendorRepository`.
- **Options Pattern:** Strongly-typed configuration binding via `RiskWeightsOptions` and `RiskThresholdOptions`.

3 Datasets Seed Data

The application loads predefined datasets from JSON files located in the API project:

- **Vendor Data:** `src/VendorRiskScoring.API/Data/SeedData/SampleVendorData.json`
- **Risk Matrix:** `src/VendorRiskScoring.API/Data/SeedData/RiskFactorMatrix.json`

4 Risk Calculation Rules & Extensible Matrix Utilization

The Vendor Risk Scoring system is designed as an open-ended, highly extensible framework. The original documentation did not explicitly define the exact mathematical mapping between all vendor data attributes and every specific value in the risk matrix. Therefore, the current implementation provides a simplified yet highly scalable baseline. As further business analysis dictates new mappings, the modular architecture allows for the seamless injection of new rules without altering the core engine.

4.1 1. Financial Risk Rule (`FinancialRiskRule`)

The financial risk evaluation is based on the `FinancialHealth` metric. Since the exact correlation between the health score and all three financial matrix areas was not explicitly provided, the system currently anchors on the `lowCashFlow` matrix entry to ensure extensibility.

- **High Financial Health (> 80):**
 - **Score:** 0.0 (No financial risk).
- **Low Financial Health (< 50):**
 - **Calculation:** The base risk multiplier is 1.0. The system calculates the average of the three triggered conditions under the `lowCashFlow` matrix entry and multiplies it by 1.0.
- **Medium Financial Health (between 50 and 80):**
 - **Calculation:** A proportional risk factor is applied based on the formula:

$$riskFactor = \frac{80.0 - FinancialHealth}{30.0}$$

This factor (e.g., 0.6) is then multiplied by the average value obtained from the `lowCashFlow` matrix entry.

4.2 2. Operational Risk Rule (OperationalRiskRule)

Operational risk directly utilizes vendor data such as `SlaUptime` and `MajorIncidents`. When a threshold is breached (e.g., SLA drops or incidents occur), the system fetches the corresponding risk category from the matrix (e.g., `slaDrop`) and calculates the average of its associated risk triggers to determine the penalty.

4.3 3. Security & Compliance Risk Rule (SecurityComplianceRiskRule)

Although the Security and Compliance domains are inherently interdependent, they are defined in separate matrices. The system evaluates them together and calculates the final category score by taking the average of their combined triggered risks.

- **Certifications:** Due to unspecified mappings for all certificate types in the original requirements, the system currently only validates the presence of the `ISO27001` certificate. If missing, it fetches the average of the three related risks in the matrix and applies a multiplier of 1.0. The architecture is built so that adding subsequent certificate checks (e.g., `SOC2`, `PCI-DSS`) is trivial.
- **Documents:** Validates operational dependencies like pentest reports, privacy policies, and contracts against their respective matrix averages.

4.4 4. Final Score Aggregation

The final risk score is not derived from hardcoded examples but calculated dynamically based on the matrix correlations. The aggregated score from each domain is multiplied by the configured weight formulations (e.g., Financial 0.4, Operational 0.3, SecurityCompliance 0.3) to compute the final, realistic vendor risk score.

5 Bonus Features & Enhancements

5.1 1. Risk Leaderboard

As an extended business capability of the API, a **Leaderboard** endpoint (`GET /api/vendor/leaderboard`) has been implemented to provide a comprehensive, aggregated overview of all vendor risk profiles.

- **Sorting Mechanism:** The leaderboard evaluates all registered vendors and sorts them in **ascending order** based on their final `RiskScore`. This mathematical ordering ensures that the most reliable and secure vendors (e.g., score 0.0, `Low` risk) appear at the top of the list, while the most critical and vulnerable vendors are pushed to the bottom.
- **Redis Cache Integration for Performance:** Generating a leaderboard for a large dataset of vendors can be a CPU-intensive operation for the rule engine. To mitigate this, the Leaderboard seamlessly integrates with the **Distributed Redis Cache**. When the leaderboard is generated, the system checks the cache for each vendor's risk profile. If a vendor was evaluated recently, the engine pulls the score directly from memory (Cache Hit) in milliseconds, bypassing the database and the matrix calculations.

6 API Endpoints & Request/Response Examples

This section illustrates sample payloads and responses for the core endpoints exposed by the Vendor Risk Scoring API.

6.1 1. Create a New Vendor

Registers a new third-party vendor into the PostgreSQL database.

- **Endpoint:** POST /api/vendor

- **Request Body:**

```
1 {
2   "id": 16,
3   "name": "Aka Trade",
4   "financialHealth": 90,
5   "slaUptime": 97,
6   "majorIncidents": 0,
7   "securityCerts": [
8     "ISO27001",
9     "PCI-DSS"
10  ],
11  "documents": {
12    "contractValid": true,
13    "privacyPolicyValid": true,
14    "pentestReportValid": true
15  }
16 }
```

- **Response (201 Created):**

```
1 {
2   "id": 16,
3   "name": "Aka Trade",
4   "financialHealth": 90,
5   "slaUptime": 97,
6   "majorIncidents": 0,
7   "securityCerts": [
8     "ISO27001",
9     "PCI-DSS"
10  ],
11  "documents": {
12    "contractValid": true,
13    "privacyPolicyValid": true,
14    "pentestReportValid": true
15  }
16 }
```

6.2 2. Evaluate Vendor Risk

Evaluates a specific vendor's risk profile (e.g., Vendor ID 5) using the strategy pattern rules and factor matrices.

- **Endpoint:** GET /api/vendor/5/risk

- **Response (200 OK):**

```
1 {
2   "riskScore": 0.82,
3   "riskLevel": "High",
4   "reason": "Financial Health is less than 50., SLA below 95%, more than 2
      major incidents, missing ISO27001 certification, No Pentest Report,
      Expired Privacy Policy and Expired Contract significantly impact the
      financial, operational and security compliance risk levels, resulting
      in a High overall risk score."
5 }
```

6.3 3. Get Risk Leaderboard (Ascending)

Retrieves all registered vendors evaluated and sorted in **ascending order** by their risk score (from the lowest risk/most reliable to higher risk levels). Below is a truncated sample of the leaderboard:

- **Endpoint:** GET /api/vendor/leaderboard

- **Response (200 OK):**

```
1 [
2
3   {
4     "vendorId": 14,
5     "riskScore": 0,
6     "riskLevel": "Low",
7     "reason": "Financial Health is more than 80. and All security and
      compliance checks passed. significantly impact the financial and
      security compliance risk levels, resulting in a Low overall risk score
      ."
8   },
9   {
10    "vendorId": 10,
11    "riskScore": 0.06,
12    "riskLevel": "Low",
13    "reason": "Financial Health between 50 and 80. and All security and
      compliance checks passed. significantly impact the financial and
      security compliance risk levels, resulting in a Low overall risk score
      ."
14  },
15  {
16    "vendorId": 6,
17    "riskScore": 0.36,
18    "riskLevel": "Low",
19    "reason": "Financial Health between 50 and 80., missing ISO27001
      certification and No Pentest Report significantly impact the financial
      and security compliance risk levels, resulting in a Low overall risk
      score."
20  }
21 ]
22 ]
```